

Завдання на підсумковий проєкт модуля 1

Модуль 1. Java Syntax
Рівень 28, Лекція 1

ВІДКРИТА

Криптологія, криптографія та криптоаналіз

Перейдемо до аналізу теорії, яка знадобиться тобі під час написання підсумкового проєкту. Давай дізнаємося більше про криптологію та її складові. А зараз — більше про шифр, який ти будеш використовувати під час написання підсумкового практичного проєкту.

1. Криптологія та її складові

Криптологія — це область знань, до якої входить:

- **Криптографія** (наука про шифри).

Предмет вивчення криптографії — шифрування інформації для її захисту від несанкціонованого доступу. Такою інформацією може бути текст, цифрове зображення, звуковий сигнал тощо. Під час шифрування утворюється зашифрована версія інформації (даних), яку називають шифротекстом, закритим текстом, криптограмою.

- **Криптоаналіз** (методи розкриття цих шифрів).

Криптоаналіз вивчає методи розкриття шифрів та способи їх застосування. Тобто виконує зворотне завдання: вивчає способи перетворити зашифровану інформацію на відкриту.

2. Криптографічний ключ

Ключ — це набір даних, за допомогою якого виконується шифрування та розшифрування інформації. Успішність дешифрування залежить від ключа, що використовується. Якщо з будь-якої причини втрачено доступ до нього, розшифрувати дані буде неможливо.

Обсяг інформації, що зберігається у криптографічних ключах, вимірюють у бітах. А це означає, що у криптографічного ключа є **довжина**. Максимальна надійність шифрування забезпечується за довжини від 128 біт.

Різновиди криптографічних ключів:

1. Симетричні (секретні). Їх використовують у алгоритмах симетричного типу. Основне призначення — зворотне чи пряме криптографічне перетворення (шифрування/дешифрування, перевірка коду автентифікації повідомлення).
2. Асиметричні. Застосовуються у шифрувальних алгоритмах асиметричного типу (наприклад, під час перевірки електронного цифрового підпису).

Ми будемо працювати із симетричним алгоритмом шифрування, тому не вдаватимемося в зайві подробиці.

3. Алфавіт у криптографії

Алфавіт — це скінченна множина символів, які використовують для кодування інформації символів.

4. Підходи до криптоаналізу

Існує багато різних підходів та методів до криптоаналізу (злому шифрів).

Опишемо найпростіші з них:

1. **Brute force** (брутфорс, пошук грубою силою) — перебирання ключів, яке виконується доти, доки не знайдемо придатний. Плюс методу полягає у простоті, мінус — у тому, що він не придатний для шифрів, які використовують велику кількість можливих ключів.
2. **Криптоаналіз на основі статистичних даних** — у разі такого підходу збирається статистика щодо входження різних символів у зашифрованому тексті, а потім для їх розшифрування використовуються статистичні дані про частоту входження у відкритий текст різних символів.

Наприклад: ми знаємо, що використання букви “П” у текстах становить 8 %. Аналізуючи зашифрований текст, ми шукаємо символ, який зустрічається у відсотковому співвідношенні таку саму кількість разів, і робимо висновок, що це буква “П”.

Мінус цього підходу — залежність від мови, авторства тексту та його стилістики.

5. Шифр Цезаря

Це один із найпростіших і найвідоміших методів шифрування. Назвали його, звісно ж, на честь імператора Гая Юлія Цезаря, який застосовував його для таємного листування з генералами.

Шифр Цезаря — це шифр підставлення: у ньому кожен символ у відкритому тексті замінюють на символ, який перебуває на певному постійному числі позицій ліворуч або праворуч від нього в алфавіті.

Допустимо, ми встановлюємо зсув на 3. У такому випадку:

- А заміниться на D
- В заміниться на Е
- С заміниться на F

Це мінімум теоретичних даних, які знадобляться тобі для виконання підсумкового проєкту. Переходимо до опису завдання!

Можна прочитати [лекцію CS50](#)

Підсумковий проєкт до модуля Java Syntax. Пишемо криптоаналізатор

Завдання: написати програму, яка працює із шифром Цезаря.

За основу криптографічного алфавіту візьми всі літери Англійського алфавіту.

Технічне завдання

1. Кінцева програма повинна бути зібрана у форматі jar. Сам jar файл розмістити в `releases` на github.

2. Програму можна запустити із консолі передавши аргументи.

- Можете вносити аргументи в Run configuration > program arguments
- Запуск програми відбувається з такими аргументами:
- `command filePath key` - саме в такому порядку.
 - Запуск самої з консолі програми буде виглядати як `java -jar myApp.jar command filePath key`
 - `command` - три доступні варіанти: `[ENCRYPT, DECRYPT, BRUTE_FORCE]`
 - `filepath` - абсолютний(повний) шлях до файлу, який кодується.
 - `key` - ціле число, ключ для зсуву по алфавіту.
- Якщо в шляху до файлу jar або до текстового файлу є пробіли або спец. символи `(* / \ $ % & # @ ! ( ) { } [ ] , ; ' " < > | ^ ~ . ) ` ,`,

то вставляйте такий шлях у подвійні лапки.

```
java -jar "c:/My Project/target/my App.jar" ENCRYPT "folder
name/textFile1.txt" 20
```

- У разі передачі `ENCRYPT/DECRYPT` `key` обов'язковий.

3. Результатом роботи програми в папці з початковим файлом повинен з'явитися файл з тим самим ім'ям, що і вхідний файл, але з поміткою `[ENCRYPTED] / [DECRYPTED]` в залежності від виконаної операції.

4. Зміст файлу повинен бути закодований/декодований використовуючи шифр Цезаря.

- Зсув по алфавіту має бути циклічним.
- Якщо ключ більше ніж кількість літер в алфавіті, то дойти до кінця алфавіту і почати спочатку.

5. Кодуються лише літери англійського алфавіту (великі та малі).

- Можна також кодувати `'.' , ',' , '<' , '>' , '"' , '\'' , ':' , '!' , '?' , ''`. Інші символи залишаються незмінними.
- Майте на увазі, що при кодуванні символів шифр стає "надійнішим", але це вже буде модифікований шифр Цезаря.

6. Після розшифрування текст має максимально зберегти оригінальне форматування (пробіли, відступи, перенесення на наступний рядок, знаки, великі та малі літери). В ідеалі текст не має взагалі відрізнатись.

7. Програма повинна використовувати один і той же ключ для коректного кодування та декодування файлу.

8. Програма має мати режим `brute-force` для автоматичного підбору ключа під зашифрований текст та його розшифрування.
9. Код програми та готовий зібраний файл `jar` розмістити на GitHub.
10. Написати про проєкт коротко в `readme`. Наприклад:
 - Що вийшло зробити.
 - Що НЕ вийшло зробити з основних вимог.
 - Особливості проєкту.
 - Які цікаві рішення реалізовані.
 - На що варто звернути увагу ментором при перевірці.

Приклади використання:

1	<code>java -jar c:/MyProject/target/myApp.jar ENCRYPT folder/textFile</code>
2	<code>java -jar c:/MyProject/target/myApp.jar DECRYPT folder/textFile</code>

Рекомендації

1. Починайте з основних вимог. Шифрування/дешифрування, читання/запис файлів.
2. **Бажано** створювати окремі класи для різного функціоналу. Наприклад:
 - `FileService` - для читання/записування файлів.
 - `CaesarCipher` - для шифрування/розшифрування тексту.
 - `CLI` - для взаємодії з користувачем.
 - `Runner` - для вирішення в якому режимі запускати програму (CLI чи працювати вже з отриманими аргументами)
3. Пишіть зрозумілі назви методів та змінних.
4. Старайтесь писати зрозумілий для читання код.
5. Пишіть маленькі методи (до 20 лінійок/рядків вважається хорошим тоном)
6. Краще не використовувати статичні методи та змінні повсюди. Використовуйте об'єкти.
7. Не забувайте про модифікатори доступу.
8. Використовуйте пакети.
9. Чим менше коду, тим краще =)
10. Для збереження форматування можна використовувати алфавіт такого формату:

`'A', 'B', 'C', ..., 'a', 'b', 'c', ..., '!', '.', '?', ...`
11. Добре протестуйте різні сценарії використання перед здачею.

Додаткові завдання

Не є обов'язковими.

1. **Поліглот.** Додати підтримку кодування тексту Українською мовою. **Easy task**
2. **Алгоритм "креатор".** Визначати автоматично який алфавіт використовувати для тексту (укр/англ). **Easy task**
3. **Поціновувач зручності.** **Medium task**
 - Зробити можливим взаємодіяти з програмою використовуючи командний рядок (CLI).
 - Зчитувати команди та параметри сканером.
 - Якщо програма запущена з переданими аргументами, то виконувати цю команду одразу, не запускаючи CLI.
4. **Частотний аналіз(Криптограф)** **Hard task**
 - У разі передачі команди `BRUTE_FORCE` передавати `key` не є обов'язковим, замість нього можна передавати посилання на файл для частотного аналізу тексту.
 - `command` `filePath` `filePathForStaticAnalysis`
 - Додати до імені файлу значення знайденого ключа.
5. **Front-end.** GUI (Swing, JavaFX) **Hard task**

Приклади використання

```
1 java -jar c:/MyProject/target/myApp.jar bruteForce folder/textF
```

На що ментор буде звертати увагу при перевірці

1. Чітке дотримування вимог в данному ТЗ (Технічному завданні).
2. Простота коду.
3. Правильне використання ООП (опціонально).
4. Цікаві рішення.
5. Додаткові завдання.

Корисні відео

1. [Цілі та завдання проєкту](#)
2. [Шифр Цезаря](#)
3. [Розбір проєкту. Частина 1](#)
4. [Розбір проєкту. Частина 2. Brute force, статистичний аналіз](#)

Проект перевіряється під час проходження його групою

[< Попередня](#)[Карта квестів >](#)

Коментарі

[популярні](#) [нові](#) [старі](#)

Oleksandr Mentor

Введіть текст коментаря



ДЛЯ ЦІЄЇ СТОРІНКИ НЕМАЄ КОМЕНТАРІВ.

НАВЧАННЯ

[Курси програмування](#)

[Допомога із задачами](#)

[Передплати](#)[Задачі-ігри](#)

СПІЛЬНОТА

[Користувачі](#)[Статті](#)[Форум](#)[Чат](#)[Історії успіху](#)[Дії](#)

КОМПАНІЯ

[Про нас](#)[Контакти](#)[Відгуки](#)[FAQ](#)[Підтримка](#)**RUSH**

JavaRush — це інтерактивний онлайн-курс вивчення Java-програмування з нуля. Він містить 1200 практичних задач із перевіркою розв'язання одним клацанням, необхідний мінімум знань із теоретичних основ Java, а ще мотивувальні «фішки», які допоможуть пройти курс до кінця: ігри, опитування, цікаві проєкти й статті про ефективне навчання та кар'єру Java-девелопера.

ПІДПISУЙТЕСЬ

МОВА ІНТЕРФЕЙСУ

Українська



DOWNLOAD APP



Програмістами не народжуються © 2026 JavaRush